CSA17 Artificial Intelligence – Scenario Based Assignment (CO2)

Question 1: Emergency Medicine Drones (Greedy Best-First vs A*)

Scenario understanding. The drone searches a graph where nodes are locations, edges are routes, and edge cost varies with terrain and weather. Paths can be blocked at any time, and the only heuristic is straight-line (Euclidean) distance h(n). Goals: minimum delay and minimum risk.

i. Greedy Best-First Search (GBFS)

GBFS expands the node with the smallest h(n) only, so f(n) = h(n). The drone always flies toward the node that looks closest to the hospital or destination in a straight line.

Strengths

- Very fast and low computation, since it ignores path cost already spent.
- Expands few nodes when the heuristic is good, which suits urgent decisions.
- Low memory use compared with A*.

Weaknesses under uncertainty

- Ignores terrain and weather cost, so it may choose a short-looking route through a flooded or high-wind zone.
- Not optimal and not complete on graphs with loops unless a visited list is used.
- Can be trapped by a dead end: a straight-line-close node may be behind a blocked river crossing.
- Straight-line distance does not know about blocked paths, so it is misled when roads fail.

ii. A* Search

A* uses f(n) = g(n) + h(n).

- g(n) = actual accumulated cost from the start, including terrain and weather penalties.
- h(n) = estimated cost remaining (straight-line distance).

g keeps the drone from choosing cheap-looking but expensive paths, and h pulls the search toward the goal. If h is admissible (never overestimates; straight-line distance is) and consistent, A* is complete and optimal. Variable costs are handled naturally because they are added into g. A blocked edge is given infinite cost, so A* routes around it.

Balance: when h dominates, A* behaves like GBFS (fast, greedy). When g dominates, it behaves like Uniform Cost Search (safe, slow). A* sits between them.

iii. Impact of heuristic accuracy

Heuristic quality	GBFS	A*
Accurate (close to true cost)	Near-direct path, very fast	Expands very few nodes, optimal
Underestimates a lot (e.g., h=0)	Becomes random/blind	Degrades to Uniform Cost Search; still optimal but slow
Overestimates (inadmissible)	Little effect on optimality (it is not optimal anyway), but can mislead	Loses optimality guarantee, may return risky or longer path

In this scenario straight-line distance underestimates real cost because terrain and weather add cost, so A* stays optimal but explores more. GBFS is more harmed because it relies on h alone.

iv. Effect of dynamic changes (blocked paths)

- A plan computed earlier may become invalid when an edge is blocked, so the drone must replan.
- GBFS replans quickly, but may repeatedly walk into new blockages, so its path can become very poor.
- A* replans at higher cost because g-values must be recomputed, but its result remains cost-aware.
- Efficient variants: **D* Lite / Lifelong Planning A*** reuse the previous search and update only affected nodes, which suits dynamic updates. Another option is **Anytime/Weighted A*** (f = g + w·h) which returns a quick answer and then improves it.
- Partial updates mean the drone's map is incomplete, so periodic replanning is needed (receding-horizon replanning).

v. Recommendation

Recommended: A* (preferably with dynamic replanning, i.e., D* Lite or Weighted A*).

- **Real-time:** A* with an informative heuristic is fast enough; Weighted A* can trade a little optimality for speed.
- **Optimality:** includes terrain and weather cost, so it finds the least-delay route; GBFS gives no guarantee.

- **Safety:** high-risk cells can be given large costs, so A* naturally avoids them; GBFS cannot see cost.
- **Dynamic environment:** D* Lite reuses earlier work when paths close.

GBFS can be used only as a quick fallback when computation time is almost zero.

Question 2: Smart City Traffic Signal Optimization

Formulation as an optimization problem

- **State:** the set of signal timings (green-phase durations and offsets) for all intersections.
- **Objective function:** minimize f(x) = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion index), where w1+w2+w3 = 1.
- **Neighbours / moves:** change one intersection's green time or offset by a small amount.
- **Constraints:** minimum and maximum green time, pedestrian crossing time, safe phase ordering.

Why traditional search is inefficient. With n intersections and k timing options each, there are k^n states, which is astronomically large. BFS, DFS and even A* cannot enumerate this space. There is no fixed goal state, only a "better or worse" objective. The traffic changes continuously, so a stored solution becomes outdated. Path cost is irrelevant here; only the final configuration matters, which is the setting for local search.

i. Hill Climbing

Start from the current timing plan, evaluate neighbours (small timing changes), move to the best neighbour that improves the objective, and stop when no neighbour is better.

Advantages: low memory, fast, easy to run continuously, works with a huge space.

Limitations: gets stuck in local optima, plateaus and ridges; no backtracking; depends on the starting point; no guarantee of global optimum; sensitive to noisy traffic data.

ii. Local maxima, plateaus and ridges (real-world meaning)

- **Local maximum (local minimum of cost):** a timing plan where every small change makes things worse, e.g., one intersection is perfectly tuned but a different, coordinated "green wave" over many intersections would be far better. The system believes it is done.
- **Plateau:** a flat region where small changes in timings do not change total waiting time (e.g., light traffic at night). The algorithm has no direction and wanders or stops.

- **Ridge:** the best improvement needs several intersections to change together (e.g., shifting offsets of a whole corridor), but single-intersection moves each look worse. Ridges require coordinated moves that simple neighbours cannot make.

iii. Overcoming the limitations

Simulated Annealing (SA). Accepts a worse move with probability e^(−ΔE/T). At high temperature T it explores widely; as T cools it behaves like hill climbing. This lets the system escape local optima, such as the single-intersection trap above. It needs a cooling schedule; with a slow schedule it converges to the global optimum with high probability. It is simple and has low memory use.

Genetic Algorithms (GA). Keep a population of timing plans (chromosomes: the timing vector), evaluate fitness (the inverse of waiting time + fuel + congestion), then apply selection, crossover (combine good corridors from two plans) and mutation (random tweaks). Crossover can make large coordinated jumps, which helps with ridges, and the population gives diversity, which helps with plateaus and local optima.

iv. Recommendation

Recommended: a hybrid, with Genetic Algorithm (or population-based search) as the main optimizer and Simulated Annealing / hill climbing for fine-tuning, re-run periodically on a rolling window of current traffic.

- **Scalability:** GA evaluates populations in parallel (each fitness evaluation can be run on a traffic simulator on separate cores). It handles hundreds of intersections better than a single-trajectory method. SA alone is lighter if computing resources are limited.
- **Adaptability:** the previous best population is a warm start when traffic changes, so the system adapts continuously rather than starting over. Use the current plan as the initial seed and re-optimize incrementally.
- **Robustness:** diversity avoids local traps; hill climbing/SA refines locally in real time.

If only one method must be chosen, choose **Genetic Algorithm** for global coordination, or **Simulated Annealing** where simplicity and low compute matter.

Question 3: Autonomous Mars Rover (Online Search)

i. Why an online search agent

- **Offline search** computes the entire plan before acting and needs a complete model of the environment. The rover has no complete map.

- **Online search** interleaves computation and action: the agent plans a step, acts, observes, and updates. This is needed when the environment is unknown (exploration problem), when actions' outcomes are discovered only by acting, and when the cost of computing a full plan is higher than acting.
- Communication delay with Earth (several minutes) rules out remote step-by-step control, so decisions must be made onboard and in real time.

ii. Challenges of partial observability and dynamic environments

- **Partial observability:** sensors see only nearby cells; craters or rocks behind obstacles are invisible; sensor noise gives uncertain readings. The agent only holds a *belief state*.
- **Dynamic environment:** dust storms, shifting sand, changing lighting, and the rover's own wheel slippage change conditions.
- **Irreversibility and safety:** some actions (driving into a crater) cannot be undone, so the agent must avoid dead ends and keep a safe return path.
- **Resource limits:** limited battery, limited computing, and limited sensing range.
- **Localization:** the rover must estimate its own position while mapping (SLAM problem).

iii. Building and updating the internal model

1. Start with an empty or sparse map (e.g., an occupancy grid with all cells "unknown").
2. After each move, sensors (camera, lidar, wheel odometry) report nearby cells; each cell is marked free / obstacle / hazard with a confidence value.
3. Update the map and the rover's position estimate using a filter (e.g., Bayesian/Kalman update).
4. Record the result of each action in a table of *result(state, action) → new state* and cost, so repeated visits are better informed.
5. Replan using the updated map; mark visited and dead-end cells so it does not loop.
6. Continue until the goal (the sample site) is reached.

This cycle is: sense → update model → plan → act → repeat.

iv. Online search compared with uninformed and informed search

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online search (e.g., online DFS, LRTA*)
Map needed	Full model	Full model + heuristic	Only local information
Plans	Offline	Offline	Interleaved with acting

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online search (e.g., online DFS, LRTA*)
Uses heuristic	No	Yes	Yes (learned/updated, e.g., LRTA* updates h)
Handles unknown terrain	No	No	Yes
Handles changes	Must restart	Must restart	Adapts as it goes
Risk	Can compute forever	Efficient but needs known map	Possible dead ends/backtracking if actions are irreversible

Online algorithms such as **Online DFS** (explores and backtracks physically) and **LRTA*** (Learning Real-Time A*, which updates heuristic values of visited states) are designed for this setting. A* can still be *used inside* the online loop on the currently known map.

v. Most suitable approach

Recommended: an online, informed search, e.g., LRTA* or repeated A*/D* Lite on a continuously updated map, combined with a safety layer.

- **Uncertainty:** the belief map is updated after each observation, and uncertain cells get higher cost.
- **Adaptability:** replanning after each new observation handles unexpected obstacles. D* Lite reuses previous computation efficiently.
- **Safety:** hazards get very high or infinite cost; the agent keeps a margin from craters, avoids irreversible moves, and keeps a known safe return route; a reactive obstacle-avoidance layer overrides the planner in emergencies.
- **Efficiency:** the heuristic (distance to sample site) guides exploration, so it is faster than uninformed online search.

Question 4: University Exam Timetable (CSP)

i. Formulation as a CSP

Variables. One variable per exam: E = {E1, E2, ..., En}. (Optionally, a variable for the exam hall assignment of each exam.)

Domains. Each variable's domain is the set of possible (time slot, hall) pairs: D(Ei) = {(t, h) : t ∈ time slots, h ∈ halls}. Domains are reduced up front by unary constraints (e.g., removing slots when the faculty member is unavailable or when a student with accommodations needs a particular hall).

Constraints

Constraint	Type	Formal description
No student has overlapping exams	Binary	If exams Ei and Ej share any student, then time(Ei) ≠ time(Ej)
Limited exam halls	Global / capacity	At any time slot, the number of exams assigned (and the seats used) must not exceed available halls and hall capacity
Precedence (some subjects before others)	Binary	If subject A must come before B, then time(A) < time(B)
Faculty availability	Unary	time(Ei) must be in the available slots of invigilators/faculty for Ei
Special accommodations	Unary / binary	e.g., extra time, separate room, or accessible hall: hall(Ei) must belong to suitable rooms, and slots must allow extra duration

This forms a constraint graph where the nodes are exams and an edge joins two exams that conflict (share students) or are ordered. All constraints above are *hard constraints*. Preferences (e.g., spreading a student's exams over days) can be *soft constraints* with a penalty cost.

ii. Backtracking search

1. Choose an unassigned exam variable.
2. Try a value (slot, hall) from its domain.
3. Check consistency with all previously assigned variables (no overlap, capacity, precedence, availability).
4. If consistent, recurse to the next variable; if not, try the next value.
5. If no value works, **backtrack** to the previous variable and change its assignment.
6. Stop when all exams are assigned (solution) or all options fail (no solution).

Example: assign Maths → Slot 1/Hall A; Physics shares students with Maths, so Slot 1 is not allowed → try Slot 2. If Chemistry later has no valid slot, backtrack and change Physics or Maths.

Plain backtracking is correct but can be exponential, as it discovers conflicts only late.

iii. How constraint propagation (e.g., forward checking) helps

- **Forward checking:** after assigning a value to an exam, remove inconsistent values from the domains of the connected, unassigned exams. If any domain becomes empty, backtrack immediately instead of discovering the failure later.
  - Example: after assigning Maths to Slot 1, remove Slot 1 from the domains of all exams sharing students with Maths.
- **Arc consistency (AC-3):** removes values that cannot take part in any consistent pair, which propagates further than forward checking.
- **Heuristics that work with propagation:**
  - **MRV** (Minimum Remaining Values): pick the exam with the fewest legal slots first (fail-first).
  - **Degree heuristic:** pick the exam involved in the most constraints (e.g., a large core subject).
  - **LCV** (Least Constraining Value): try the slot that rules out the fewest options for neighbours.

Result: earlier failure detection, a smaller search tree, and far fewer backtracks.

iv. Real-world challenges and best strategy

Challenges

- Thousands of students and courses create dense constraint graphs and an exponentially large space.
- Conflicting constraints may make the problem unsatisfiable, so some constraints must be relaxed.
- Last-minute changes (faculty unavailable, new accommodations) require rescheduling.
- Both hard constraints and soft preferences (fairness, spacing) must be handled.

Best strategy

- Use **backtracking with MRV + degree + LCV, forward checking / AC-3**, and add **conflict-directed backjumping** where possible.
- For very large instances, use **graph-colouring-based decomposition** (exams = nodes, conflicts = edges, slots = colours) and split into independent subproblems (e.g., by department).

- Combine with **local search (min-conflicts)** or simulated annealing to repair an existing timetable when changes arrive, which is much faster than restarting.
- Treat soft constraints as an optimization on top of a feasible solution.

This hybrid of systematic search with propagation (to get feasibility) and local search (for repair and scalability) is the most scalable.

Question 5: Strategic Game AI (Minimax and Alpha-Beta)

i. How Minimax models the problem

- The game is modelled as a two-player, zero-sum, adversarial game tree: **MAX** is the AI, **MIN** is the human opponent.
- Nodes are game states, edges are legal moves, and leaves (or depth-limited nodes) have utility values from an evaluation function.
- MAX levels pick the child with the maximum value; MIN levels pick the minimum, assuming the opponent plays optimally.
- Values propagate up the tree, and the AI plays the move with the best guaranteed outcome against the best opposing play: Minimax(s) = max over actions of Minimax(result(s, a)) at MAX nodes, min at MIN nodes.
- For a real-time strategy (RTS) game, the model is used with simplifications: a limited look-ahead depth and an abstracted state.

ii. Computational challenges

- Time complexity is O(b^d) and space is O(b·d) (depth-first), where b = branching factor and d = depth. With large b (many units, many possible actions per turn) the tree explodes: for example b = 30, d = 10 gives about 30^10 ≈ 6×10^14 nodes.
- Strict time limits make it impossible to search to terminal states, so a depth cutoff and an evaluation function are needed.
- Evaluation of complex states (resources, units, map control) is itself expensive.
- In real-time games, states change continuously (not turn-based), requiring discretization.
- Partial information (fog of war) and non-determinism break plain Minimax assumptions.
- The horizon effect: the AI can miss a danger just beyond the search depth.

iii. How Alpha-Beta pruning improves efficiency

- **α** = best value MAX can already guarantee along the path; **β** = best value MIN can already guarantee.
- Initially α = −∞, β = +∞. During search, MAX updates α and MIN updates β.
- **Pruning rule:** if α ≥ β at a node, the remaining children cannot influence the final decision and are skipped.
- **Reasoning:** suppose MAX already has a move guaranteeing value 5. At a MIN node, the first child returns 3. MIN can force ≤ 3 here, which is worse for MAX than 5, so MAX will never choose this branch, and the other children of that MIN node need not be examined.
- It returns the **same result as Minimax**, with no loss of optimality.
- Complexity: best case (perfect move ordering) O(b^(d/2)), which allows searching about **twice as deep** in the same time; average case about O(b^(3d/4)); worst case O(b^d) (no pruning, with bad ordering).
- **Improvement by move ordering:** examine the most promising moves first (captures, high-value actions, killer moves, previously best move) to maximize pruning.

iv. Evaluation function design

A weighted linear function: **Eval(s) = w1·f1 + w2·f2 + ... + wn·fn**, normalized and from the AI's point of view.

Factor	Reason
Material / army strength difference (number and quality of units)	Direct measure of ability to win fights
Resource income and stockpile (gold, wood, etc.)	Economy enables future units and upgrades
Map control and territory held, vision	Controls expansion and information
Base health / key structure safety	Losing the main base means losing the game
Technology level and upgrades	Long-term strength
Unit positioning and threat/mobility (e.g., units in range of enemy)	Tactical advantage
Tempo/production capacity	Ability to replace losses

Weights can be set by expert knowledge or learned from self-play or game data (machine learning). Terminal states get ±∞ (win/loss). Features should be cheap to compute (needed at every leaf), reflect the game's strategy phase (opening, mid, end game weights differ), and be tested for balance to avoid the AI over-focusing on one factor.

v. Strategy balancing optimality and real-time performance

Recommended strategy: Alpha-Beta with iterative deepening and a time limit, plus supporting enhancements.

1. **Iterative deepening:** search depth 1, 2, 3, … until the time budget ends, and always have the best move from the last completed depth ready (an *anytime* algorithm). It also improves move ordering for the next depth.
2. **Depth-limited search with the evaluation function** at the cutoff; add *quiescence search* to avoid the horizon effect in unstable positions.
3. **Move ordering and transposition table** (cache of already-evaluated states) to prune more and avoid repeated work.
4. **Selective search / forward pruning:** limit the branching to the top-k promising actions; abstract units into groups to reduce b.
5. **Opening book / precomputed strategies** for early game, and **Monte Carlo Tree Search** as an alternative for very large or uncertain (fog of war) cases.
6. **Adaptive difficulty:** vary depth/time to match player skill.

This keeps decisions inside the time limit, near-optimal for the searched depth, and avoids the full cost of unpruned Minimax.

Rubric alignment: each answer states the scenario understanding, identifies the key issues (uncertainty, dynamic change, local optima, constraints, branching), applies the correct algorithms, and finishes with a justified recommendation.